Closing the Documented Limitations of an Agent-Harness Memory Extension

Authors: Anstar, Z.[1]*, Zeng, P.[1], and DeepSeek Harness Contributors[2]

Affiliations: [1] Superman Project, Independent Research. [2] DeepSeek Harness Contributors, open-source community.

Corresponding author: *e-mail: anstar@superman-project.org

Abstract

Agent harnesses accumulate capability faster than they retire their documented weaknesses, and a limitation recorded in a release note rarely receives the measurement that would show it closed. This paper reports the systematic closure of the eight limitations published with v1 of the Superman suite, an out-of-tree extension for DeepSeek Harness (DSH). Six were closed, one partially, and one not at all. On-device optical character recognition (OCR) through the macOS Vision framework turns image-only pages whose text layer holds zero characters into 3 411 recognised characters, 979 of them Han, and recovers 11 333 characters from a 19-page brochure whose embedded CMap is damaged. Recall was rebuilt as BM25 with $k1 = 1.2$ and $b = 0.75$ over name, body and record fields, so a rare query term now outranks a term every document carries. Conversations, images, capacity and store-time extraction moved into the memory: a fresh browser origin restores a conversation from the memory, a resent image is refetched byte-for-byte, PDFs are parsed as they are stored, and item and byte caps evict oldest-first while start-up reconciliation repairs the index. Offline verification grew from 284 to 449 checks across six suites. The browser-level DOM suite remains unconverged and semantic recall remains open because no embedding service exists on this machine; both are reported as such rather than presented as solved.

Keywords: agentic coding harness, plugin extension, optical character recognition, BM25, durable memory, compaction, offline verification

1 Introduction

An agentic coding harness is a long-running process that owns a model connection, a tool registry, a session log and a user interface. DeepSeek Harness (DSH) [1] is one, built on the Cordis plugin framework [2]; its interface is a browser application served from a loopback address, and its primary affordances are the session, the turn, the tool transcript and the diff. That design suits its task — delegating bounded engineering work to an autonomous loop — and the loop itself is well studied: interleaved reasoning and acting [3], learned tool invocation [4], and the problem of keeping a context useful as a session grows [5]. Less well studied is the maintenance question that follows any such system into its second release: what happens to the weaknesses its authors were honest enough to publish.

The v1 release of the Superman suite was an out-of-tree extension for DSH that added a conversational surface, a persistent memory and a literature-capture path without forking the harness. Its article reported the architecture, the measured behaviour of a hand-written PDF text extractor, and — in a discussion section rather than a footnote — eight limitations. Those limitations were specific: they named an input class the extractor could not read, a scoring function with a known bias, a storage location with a known failure mode, and the absence of a whole class of test. This paper is the second half of that work.

In v1's order: PDF extraction read the text layer only, so scans yielded nothing (L1); recall was a flat additive lexical score that favoured long documents (L2); history lived in the page origin's `localStorage` (L3); images in history kept a 160-pixel thumbnail and were resent as unavailable (L4); the memory grew without a compaction policy (L5); panel uploads and harness captures stored PDF bytes without text (L6); there was no automated browser-level test (L7); and recall was not semantic (L8). Six are now closed, one partially, and one could not be closed on this machine at all, with the evidence for each outcome a measurement, a check in an offline suite, or both, and the residue stated in the same section as the fix rather than in a closing paragraph.

Three subsystems make up the suite, and all three are affected. Chat mode is a conversation panel beside the agent: it calls no tools, accepts attachments, stores history and renders Markdown with mathematics. Superman is a smart folder in the sidebar into which every file or image uploaded in either mode is classified and indexed, and which both modes consult while they think. arXiv bookmarking adds a save to memory button to each search result, archiving a paper's PDF into Superman and degrading to a metadata record when the download is blocked. v2 keeps

that architecture and changes its internals; a reader who wants the design of the suite itself should read the v1 article, because this paper recapitulates only what the fixes touch.

Why should the closing of limitations be a paper? Because the work is not cosmetic: closing L1 required a second extraction engine, written in a different language and compiled on first use, wired into an upload path as an automatic fallback; closing L2 replaced the recall scoring function outright; and closing L5 added a capacity policy, an eviction path and a start-up reconciliation pass to a store whose only previous invariant was that writes were atomic. Each has a failure mode of its own and each was validated the way the original system was, by driving the shipped code with real inputs. Because the measurement matters as much as the fix: it is easy to claim a scanned document is now readable, and more useful to say that a rasterised image-only page has zero characters in its text layer and that the OCR path returns 3 411, of which 979 are Han. And because the residue is informative: L7 is a tooling problem with a known endpoint, L8 a resources problem with no evaluation available, and reporting both alongside six closed items is more useful than reporting seven closures and one apology.

The contributions are therefore the following. A closure register for the eight v1 limitations, each carrying the problem, the fix, the evidence, the status and the deployment consequence (Table 1, Table 2, Figure 1, Figure 2). On-device OCR as an automatic fallback: a self-contained Swift program that renders pages with PDFKit [6] and recognises text with the Vision framework [7], wrapped in a host bundle that compiles it on first use, called only when the text layer is empty, and degrading silently when the bundle is absent (Section 2.3). BM25 recall over weighted fields: the additive scorer replaced by BM25 [8] with $k1 = 1.2$ and $b = 0.75$, name-weighted fields, inverse document frequency over the whole memory and the recency term retained, with the returned structure and rendered digest unchanged (Section 2.4). Durable, restorable conversation state (Section 2.5). Image fidelity through the memory: a resent image is refetched byte-for-byte from the file route, falling back to an explicit note only when the item is gone (Section 2.6). A capacity policy with reconciliation: item and byte caps, oldest-first eviction of entry and file, a start-up pass that drops vanished entries and unreferenced files, a manual `prune`, and eviction and orphan counters in `stats` (Section 2.7, Figure 9). Store-time extraction on the host: a Node port of the browser PDF reader that parses a document as it is stored, with page and character caps, never failing the store (Section 2.8). A DOM harness and its honest status: a workspace-local jsdom [9] plus React [10] toolchain and an 884-line driver that mount the panels through `createRoot` into a real document, with the measured statement of what prevents it from shipping (Section 2.9). And an open item with a seam: semantic recall is not implemented, and the paper states what an embedding path would need and why the single-function seam makes that an addition rather than a rewrite (Section 2.10).

Section 2 gives the fix programme, the verification programme and the eight limitations one by one. Section 3 discusses what a closure pass teaches, the threats to validity, and the status of the two open items. Section 4 gives the methods, and Appendices A–E collect the API actions, the suite inventory, the corpus and OCR tables, the limitation register and the repository layout.

2 Results

2.1 The fix programme

v2 addresses the eight limitations v1 published. It is not a rewrite: the bundles keep their identities, the browser halves keep their contracts, and the memory keeps its directory layout. What changed is the behaviour behind four seams — the extraction path, the recall function, the history store and the capacity policy — plus one new companion bundle for optical character recognition.

The outcome is summarised in Table 2 and drawn as a matrix in Figure 2. Six items are closed: L1 (scanned and outlined PDFs), L2 (recall scoring), L3 (history durability), L4 (image fidelity in restored turns), L5 (capacity and consistency) and L6 (text for stored PDFs). One is partially closed: L7, the browser-level DOM suite, which exists and runs but is not in the published tree. One is open: L8, semantic recall, because no embedding service exists on the machine where this work was done. Figure 1 contrasts the two topologies: the v1 path, in which extraction, storage and recall all terminate at the browser, and the v2 path, in which the host half parses documents as they are stored, an optional OCR bundle reads pages that carry no text layer, images are hydrated back from the memory before a turn is sent, and a compaction policy bounds the store.

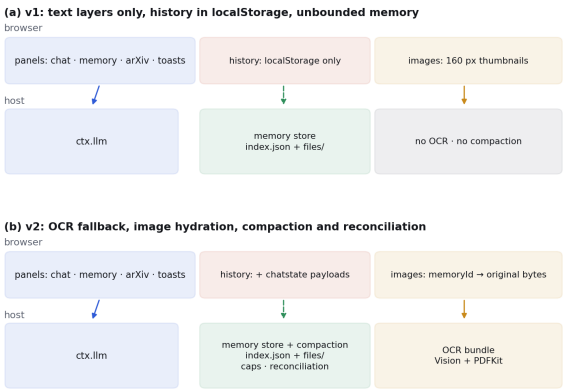


Figure 1. Extension topology before and after the closure pass. (a) v1: the browser extracts, stores and recalls, and the host half only holds the memory. (b) v2: the host half extracts text from PDFs at store time, the optional OCR bundle recognises pages with no text layer, the browser hydrates stored images before sending, and compaction and reconciliation bound and repair the store.

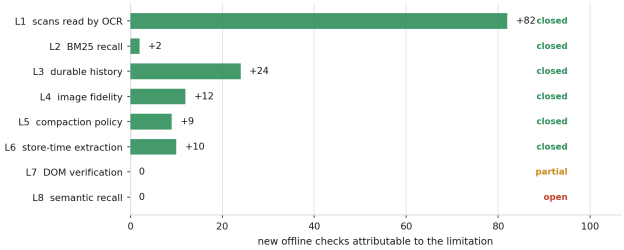


Figure 2. The eight v1 limitations as a fix matrix. Closed items (L1, L2, L3, L4, L5, L6), the partially closed DOM suite (L7) and the open semantic-recall item (L8) are shown with the verification delta each change contributed.

The deployment consequence of each fix differs, and Table 1 records it, because it determines how a user obtains the change. Client halves are plain ES modules served by the client-module graph, and the hot-reload watcher polls each bundle's modification time and size every 500 ms, so a browser-half change reaches a running page without a restart; host halves are ordinary Node modules, so adding a new bundle activates immediately but editing an already-loaded host half needs a process restart. The three browser-side fixes — the OCR fallback call, history hydration and image restoration — therefore take effect immediately, while BM25 recall, compaction with reconciliation and store-time extraction do not.

Table 1. What changed in v2, and where each change takes effect.

#	area	v1 behaviour	v2 change	takes effect
L1	scans and OCR	PDF extraction reads the text layer only; scans yield nothing	on-device OCR through macOS Vision, wired into the Chat upload path as an automatic fallback	client HMR + OCR bundle
L2	recall scoring	recall was a flat additive lexical score that favoured long documents	BM25 (k1 = 1.2, b = 0.75) with IDF over name/body/record fields plus the recency term	host, restart
L3	history storage	conversation history lived in the page origin's localStorage	every finished turn also writes a restorable JSON payload (chatstate:<id>, ≤ 7 900 characters) that merges on start-up	client HMR
L4	image fidelity	images in history kept only a 160 px thumbnail and resent as unavailable	an image remembers the memory item id of its bytes; a resend fetches the original from the file route	client HMR
L5	compaction	the memory grew without a compaction policy	maxItems 2 000 and maxBytes 512 MiB caps, oldest-first eviction, start-up reconciliation, prune, eviction counters	host, restart
L6	store-time extraction	panel uploads and Harness captures stored PDF bytes without text	superman/pdf.js, a Node port of the browser extractor, parses on store (200-page / 400 000-character caps)	host, restart
L7	DOM verification	no automated browser-level (DOM) end-to-end test	a workspace-local jsdom + React toolchain and tools/dom-check.mjs mount the panels with createRoot; environment differences remain	toolchain, not shipped
L8	semantic recall	recall is not semantic	no embedding service exists on this machine; retrieval stays lexical (now BM25), the seam is recall(q)	—

Table 2. Limitation, fix, evidence and status for the eight v1 items.

#	v1 limitation	v2 fix	evidence	status
L1	PDF extraction reads the text layer only; scans yield nothing	on-device OCR through macOS Vision, wired into the Chat upload path as an automatic fallback		closed
L2	recall was a flat additive lexical score that favoured long documents	BM25 (k1 = 1.2, b = 0.75) with IDF over name/body/record fields plus the recency term		closed
L3	conversation history lived in the page origin's localStorage	every finished turn also writes a restorable JSON payload (chatstate:<id>, ≤ 7 900 characters) that merges on start-up		closed
L4	images in history kept only a 160 px thumbnail and resent as unavailable	an image remembers the memory item id of its bytes; a resend fetches the original from the file route		closed
L5	the memory grew without a compaction policy	maxItems 2 000 and maxBytes 512 MiB caps, oldest-first eviction, start-up reconciliation, prune, eviction counters		closed
L6	panel uploads and Harness captures stored PDF bytes without text	superman/pdf.js, a Node port of the browser extractor, parses on store (200-page / 400 000-character caps)		closed

#	v1 limitation	v2 fix	evidence	status
L7	no automated browser-level (DOM) end-to-end test	a workspace-local jsdom + React toolchain and tools/dom-check.mjs mount the panels with createRoot; environment differences remain		partial
L8	recall is not semantic	no embedding service exists on this machine; retrieval stays lexical (now BM25), the seam is recall(q)		open

The size of the change is modest and unevenly distributed. Bundle JavaScript grew from 7 187 lines across six files to 8 869 lines across eight: the browser half of chat mode grew by 264 lines for hydration, OCR fallback and restorable payloads; the memory's host half grew by 242 lines for BM25, compaction, reconciliation and store-time extraction; and two files are new, the host-side PDF extractor at 799 lines and the OCR bundle at 383 lines. The self-contained Swift program the OCR bundle compiles is under 220 lines, while the memory panel and the arXiv bundle did not change. Verification grew more than the code: v2 holds 449 checks in six offline suites against 284 in the five suites of v1. The chat pipeline suite grew from 445 to 823 lines for 139 checks, the memory host suite from 199 to 501 lines for 71 checks, and a new suite of 407 lines holds 71 checks for the OCR bundle. All 449 are green on this machine.

2.2 The verification programme

The method is described in Section 4.6: the browser halves load under a stub module loader with a stub React that invokes function components directly, the host halves run against a fake Cordis context, and real inputs are used wherever they exist — a fourteen-document PDF corpus, real encoding bytes, generated PDFs carrying FlateDecode and Type0 text layers, and the real Swift source compiled by the real compiler. That proves that the shipped parsing, scoring, framing, storage and element trees behave as asserted; it does not prove pixel-level appearance, real-browser layout, real provider output, or the harness's own authentication fence.

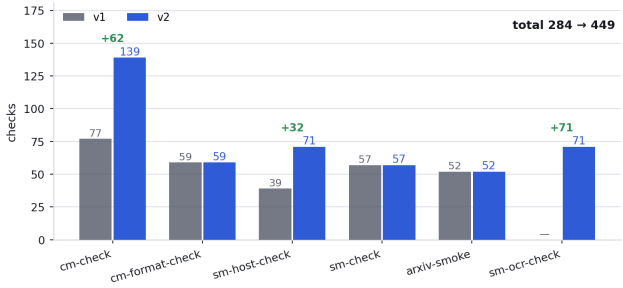


Figure 3. Verification checks per suite, v1 against v2, with the totals annotated.

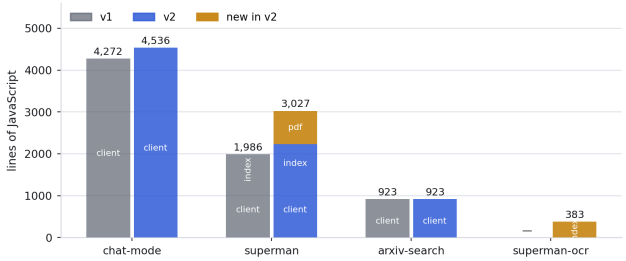


Figure 4. Code growth per module between v1 and v2, with the two new files highlighted.

Table 3. Source inventory of the v1 and v2 trees.

file	v1 lines	v2 lines	delta
`chat-mode/client.js`	4,017	4,281	+264
`chat-mode/index.js`	255	255	+0
`superman/client.js`	1,320	1,320	+0
`superman/index.js`	666	908	+242
`superman/pdf.js`	—	799	new
`superman-ocr/index.js`	—	383	new
`tools/ocr-pdf.swift`	—	214	new
`arxiv-search/client.js`	784	784	+0
`arxiv-search/index.js`	139	139	+0
total (module .js)	7,181	8,869	+1,688

Table 4. Verification suite inventory: what each suite drives and how many checks it holds.

suite	what it drives	v1	v2	lines
`chat-mode-check.mjs`	pipeline: route, SSE, history, attachments, memory wiring	77	139	823
`chat-mode-format-check.mjs`	rendering and decoding: LaTeX→MathML, Markdown, encodings	59	59	263
`superman-host-check.mjs`	memory host half: extraction, BM25 recall, caps, reconciliation	39	71	501
`superman-check.mjs`	memory panel: entry, folders, preview, upload, notices	57	57	348
`arxiv-smoke.mjs`	arXiv bundle including the bookmark path	52	52	450
`superman-ocr-check.mjs`	OCR bundle: route, envelope, compile-on-first-use, real engine	—	71	407
total		284	449	2,792

Figure 3 shows the distribution and Figure 4 the code growth; Table 3 lists the source inventory and Table 4 the suites. The largest proportional growth is not in the chat pipeline, where the OCR fallback and hydration were added, but in the memory host suite, which nearly doubled from 39 to 71 checks because compaction, reconciliation and BM25 each needed adversarial cases: a store burst that must leave `index.json` valid, a byte cap that must hold after every store, an orphan file that must be deleted, an entry whose file vanished that must be dropped, and a file an entry still references that must never be deleted. That distribution is the honest signature of this pass: most of the new verification is about the store's new invariants rather than about new features.

2.3 L1: on-device OCR for documents with no text layer

The original problem. The v1 reader recovers text by parsing the PDF itself: it scans the object table, inflates `FlateDecode` streams, maps glyph codes through each font's `/ToUnicode` CMap and replays content-stream text operators [11], [12]. That is sufficient for a born-digital document and useless for a scan, where the page is a bitmap and the content stream draws no text. Nine of the fourteen documents in the v1 corpus were of that kind — vector art and scanned pages from one vendor — and the reader refused them honestly with a `no-text` note rather than emitting noise. The limitation was the boundary of a text-layer reader, and the only way past it is to look at the pixels. Nor is it a defect in the documents: an archival profile such as PDF/A [13] may legitimately carry a scanned image with no text layer, so `no-text` describes a valid document that this reader cannot serve.

The fix. v2 adds a second extraction engine and a place to call it from. The engine is a self-contained Swift program, `tools/ocr-pdf.swift`, linking PDFKit [6] and the Vision framework [7] and nothing else. For each page it renders the media box into an 8-bit RGB bitmap at two device pixels per PDF point, filling white first because a transparent bitmap reads badly for some glyphs; a page whose longest edge would exceed 4 000 pixels is rendered down so that one page cannot exhaust memory; and a page that refuses a bitmap context falls back to the thumbnail renderer. The bitmap goes to a `VNRecognizeTextRequest` configured for `.accurate` recognition, with languages `zh-Hans` and `en-US` and language correction enabled. Vision's order is close to reading order but not guaranteed, so observations are sorted by the line's vertical centre and then its left edge, and each page is emitted as a `## page N` marker followed by its lines as soon as it is recognised, so a caller that kills the process on a timeout still keeps finished pages. The exit status is non-zero when the document cannot be opened or no page could be rendered.

Compiling that program on a user's machine is the deployment problem. The OCR bundle, `@local/dsh-superman-ocr`, exposes one authenticated route, `POST /api/superman-ocr/api`, with two actions: `health` reports whether a runnable binary exists, which path it is, whether this process compiled it and the version it reports; `ocr` takes a base64 PDF, an optional name and an optional page budget, and returns the recognised text, the number of page markers and the true character count before truncation. The binary is produced by `swiftc` on first use into the bundle's work directory, from a staging path renamed into place and marked executable only after a successful exit. A compiled binary newer than its source is adopted rather than rebuilt; within a process, concurrent callers share one compile promise, so a burst of uploads triggers one `swiftc` invocation; and a failed or timed-out compile installs nothing and is retried. Limits are explicit: 40 pages by default and 200 as a hard ceiling, a 32 MiB PDF ceiling checked on the base64 length and on the decoded bytes, 400 000 characters of returned text, and a 600-second wall-clock budget with a 300-second budget for the compile. Each call writes its PDF to a uniquely named temporary file and deletes it in a `finally` block. Error codes are stable: `bad-request` (400), `too-large` (413), `aborted` (499), `compile-failed` (500) with the compiler's standard-error tail, `ocr-failed` (502) and `ocr-timeout` (504).

The chat half calls this engine as a fallback and only as a fallback. When a PDF is attached the browser extractor runs first; if its text is empty, the composer calls the OCR route with the file's bytes and an abort timer and, on success, inlines the recognised text under an explicit marker saying the content was recognised on this machine and may contain character errors. The same text is what is stored in the memory, so recall sees it too. If the OCR bundle is absent, or the route fails, or the recognised text is empty, the composer keeps the original no-text message and reports the file as skipped; nothing throws, and no bytes are stored as if they had been read. That asymmetry is deliberate: a silent fallback that swallowed a failure would turn an honest refusal into a wrong answer.

The measurement. The decisive number is a zero. A page rasterised into an image-only PDF contains no extractable text: the reader returns 0 characters. The OCR path returns 3 411 characters from that page, 979 of them Han. A second rasterised page is the same story at a smaller scale: one character in the text layer, 1 341 characters from OCR, 1 072 of them Han. A third case isolates a different failure. A 19-page brochure has a text layer that parses — 9 423 characters — but its embedded CMap is damaged, so the layer is almost entirely wrong script: only 9 of those characters are Han. Running the OCR engine on that document returns 11 333 characters, 66 of them Han. Layer and recognition therefore disagree about what the document says, and the recognition is the one that reads the page. Figure 5 plots the pairs and Table 5 gives each case with its page count and cost.

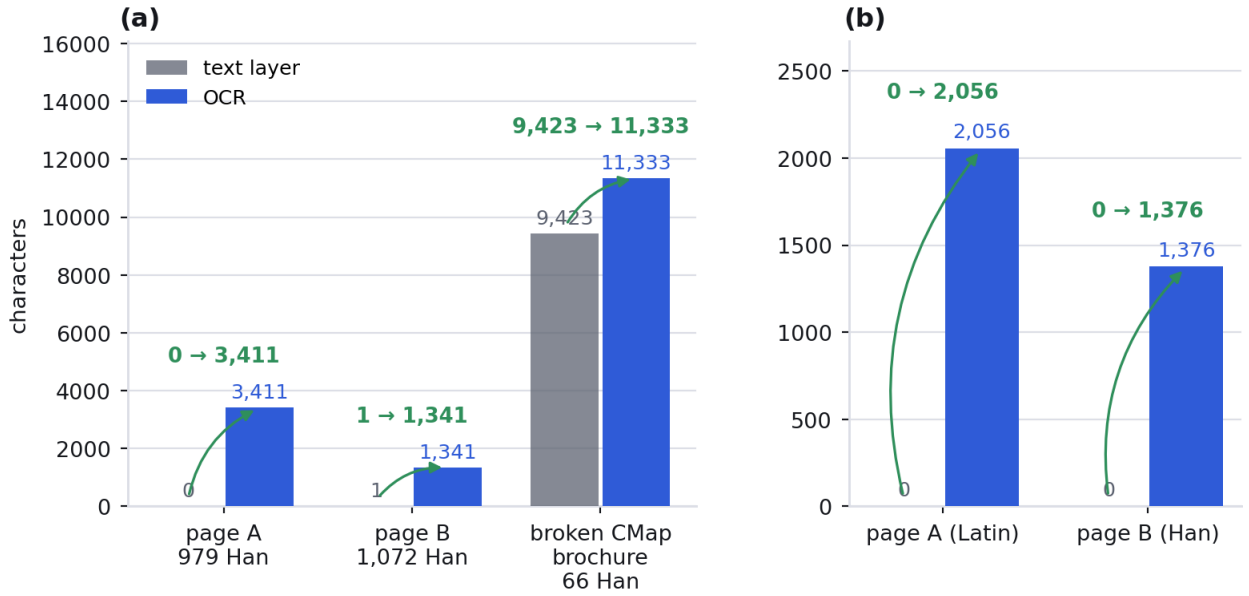


Figure 5. Text-layer yield against OCR yield for the rasterised and damaged-CMap cases, annotated with the decisive 0 → 3 411 result.

Table 5. OCR measurements per case: text-layer characters, recognised characters, Han characters, pages and time.

case	pages	layer chars	layer Han	OCR chars	OCR Han	ms
rasterised page A	1	0	0	3,411	979	—
rasterised page B	1	1	0	1,341	1,072	—
broken-CMap brochure	19	9,423	9	11,333	66	—
rasterised page A (Latin)	1	0	0	2,056	0	1,470
rasterised page B (Han)	1	0	0	1,376	992	880

One aspect of that measurement is a limit rather than a success. The automatic trigger is an empty text layer, so the 19-page brochure — whose layer is present but wrong — is not rescued by the fallback on upload; its OCR figures come from invoking the engine on the file directly. A second trigger, firing when a layer's language profile contradicts its declared encoding or when a page yields implausibly few characters for its drawing operations, would be a reasonable next step, and the sparse-layer marker the composer already emits for layers under 60 characters per page is the natural place to hang it. As implemented, the fallback fires only where the alternative is nothing.

Context: the corpus and the cost of the text-layer path. The OCR cases sit on top of the v1 corpus, which remains the reference measurement for extraction: fourteen real documents, 199 942 characters, 67 235 of them Han, 138 pages, zero replacement and C1 control characters, five documents with a text layer and nine `no-text` reports. Those nine are the zero line in Figure 6, where the OCR-recovered cases are marked; the two largest documents are 56-page and 54-page Chinese prospectuses at 90 938 and 89 764 characters, both with zero noise. Cost tracks the volume of text replayed rather than file size: medians are 34.7 ms for the 19-page brochure, 71.8 ms and 71.3 ms for the two large Chinese documents, 11.7 ms for a 7-page contract, 7.3 ms for two pages, and 0.2–0.8 ms for the nine textless vector files, which are refused early. The 19-page file is larger on disk than the 56-page one and faster to read, which is the clearest evidence that the cost is decoding work rather than I/O. Figure 7 plots the median against page count; the OCR path is not in that figure, because it runs only after extraction has failed and its cost is measured in Table 5.

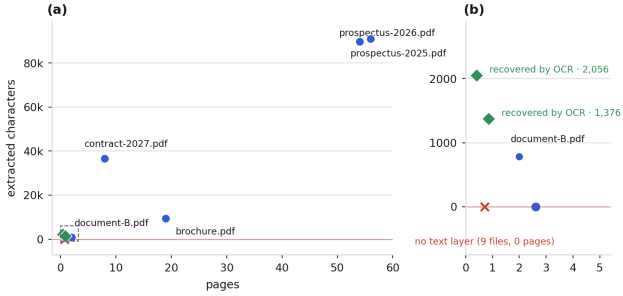


Figure 6. Extracted characters against pages for each corpus document. The nine textless files sit on the zero line; the OCR-recovered cases are marked.

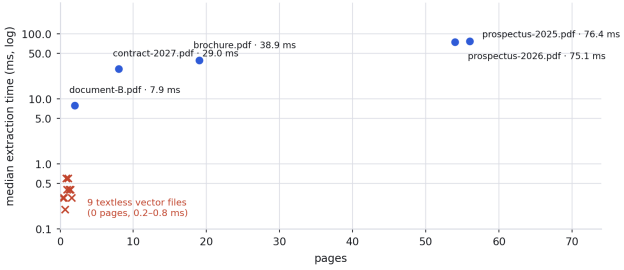


Figure 7. Median extraction time against document pages for the text-layer reader, annotated at the corpus medians.

2.4 L2: BM25 recall in place of an additive score

The original problem. v1 scored an item against a query by adding up evidence: a token in the file's name was worth 4, a token of two or more characters anywhere in the name, category or body was worth 1.5 (0.6 if shorter), a token occurring more than three times earned 0.5 more, and recency added up to 1.5 over thirty days. Two properties of that formula are wrong for a corpus of documents of unequal length. First, the score summed raw term frequencies with no length normalisation, so a long document that mentions a query term many times beats a short document that is about it: the scorer was biased towards length. Second, a term every document carries contributed as much as a term only one document carries, so a query containing a common word diluted the signal from the rare one. Both problems are standard [8], [14], [15], [16], and the standard answers are to weight a term by how surprising it is in the collection and to normalise a document's length against the collection average.

The fix. v2 replaces the additive score with BM25, $k1 = 1.2$ and $b = 0.75$. Every item and record in the memory becomes one document, represented as a weighted term-frequency map in which the name (or a record's title) weighs 3 and the body weighs 1, so the frequency $f(t, D)$ is a weighted count and the document length $|D|$ a weighted token count. Tokenisation is unchanged from v1: lowercase Latin words of two to forty characters, plus CJK characters and their bigrams, which gives Chinese queries surface coverage without a segmenter. The query is tokenised without repeats; document frequency is computed over the whole memory for each query token; the average document length is the weighted average over all documents; and a document's score is the sum over query tokens of the inverse document frequency of the token times the saturated, length-normalised term frequency:

$$\text{score}(D, Q) = \sum \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k1 + 1)}{f(t, D) + k1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \quad \text{for } t \in Q$$

$$\text{IDF}(t) = \ln\left(1 + \frac{N - n(t) + 0.5}{n(t) + 0.5}\right)$$

where N is the number of documents and $n(t)$ the number containing the token. Documents scoring zero are dropped, the survivors are sorted, and the v1 recency term is added afterwards — still capped at 1.5 over a thirty-day window, so two equally relevant documents are presented newest first without recency overriding relevance — and the list is sorted again by the sum, truncated to the requested limit of six by default and twenty at most, with a 200-character snippet centred on the earliest matching token. The shape of the reply did not change: recall returns the same `matches` array with the same fields plus a `score`, and renders the same prompt-ready digest with a header naming the number of hits. No caller was edited.

Two consequences are worth stating because they are properties rather than tuning effects. Length normalisation means that, for a fixed term frequency, a short document scores at least as high as a long one and the penalty grows with the document's length relative to the corpus average, saturating at $b = 1$. Inverse document frequency means a token present in every document contributes almost nothing: with $n(t) = N$ the numerator is 0.5 and the denominator $N + 0.5$, so the IDF is the logarithm of 1 plus a small quantity, close to zero without reaching it, which keeps the function continuous while making ubiquitous tokens irrelevant in practice.

The measurement. The memory host suite contains three documents built for this question: one containing the rare token `zebra`, one consisting of the token `the` repeated, and a filler document. Querying with both tokens — `zebra the` — must return the rare-term document first; under the v1 additive score the common token was worth as much as the rare one and the repeated document could win on frequency alone. The suite asserts the first match's identity, and separately that the score of the rare token alone exceeds the score of the common token alone. A third assertion tests the IDF function directly: the IDF of a token occurring in all three of three documents, multiplied by five, is still less than the IDF of a token occurring in one of three. All three pass, along with the older checks that recall matches file contents and not only names and that an empty question recalls nothing. Figure 8 draws the worked example, with the rank flip annotated.

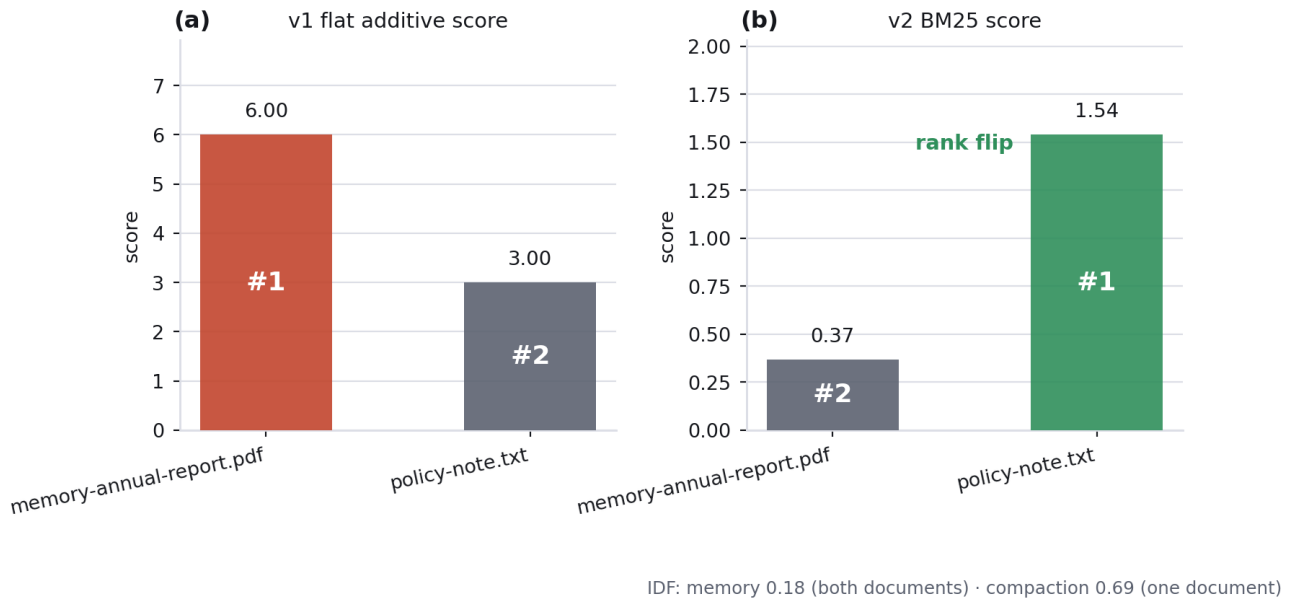


Figure 8. Worked BM25 example. Two documents and one rare query term, scored under the flat additive scheme and under BM25, with the rank flip and the inverse-document-frequency weighting annotated.

2.5 L3: conversation history that survives the origin

The original problem. v1 kept the conversation list in one JSON document under the page origin's `localStorage` [17], at the key `dsh.chat-mode.history.v1`, with caps of 40 conversations, 200 messages per conversation and 3 500 000 characters in total, 40 000 characters of attachment text and 20 000 of reasoning per message, and a 160-pixel thumbnail per image. That design has a real virtue — the fast copy lives where the rendering needs it and needs no host service — and one failure mode, which v1 stated plainly: page-origin storage is erased when site data is cleared, and it is not shared with a harness instance on another port, because a different port is a different origin.

The fix. v2 leaves the local copy where it is and adds a durable one behind it. At the end of every turn, in addition to the prose record recall searches (id `chat:<conversationId>`, the last 8 000 characters of the exchange), the composer writes a second record under the id `chatstate:<conversationId>` whose body is a restorable JSON payload. The payload declares version 1 and carries the title, update time, model and effort selection, and one entry per message with its role, text and attachment metadata: name, kind, byte size, media type, thumbnail, display detail, the memory item id for images, and up to 2 000 characters of body text for text and PDF attachments. It is written to fit one memory record, capped at 8 000 characters, so the serialiser trims to 7 900 — by dropping the oldest turns one at a time and re-serialising, and only if a single remaining turn is still too large by shortening message texts to 600 characters. The JSON is therefore always whole: an over-long conversation loses its oldest turns rather than being cut mid-object, which matters because the reader parses it.

Reading is split the same way, asynchronously on top of synchronously. On start-up the panel loads the local list and opens the conversation that was open, with no wait on the network, then asks the memory for its records, keeps those whose id begins `chatstate:`, discards any payload whose version or message array does not match, and merges what survives into the list already on screen. A memory record is adopted when the local list has no copy of that conversation or when its update time is newer; a newer local copy is never replaced. The merged list is written back to local storage so the next start is local again, and if the open conversation gained messages from the merge, the open thread is upgraded in place. Memory commands are bounded by an eight-second timer with an abort, so a memory that is not answering degrades to the local list rather than delaying it.

The measurement. The chat pipeline suite drives a client whose local storage is empty and whose stubbed memory hands back one `chatstate` record. The restored list must contain that conversation with the memory's id; the conversation must be cached locally by the merge; the status must be built from the `memoryRestored` sentence with the count substituted; the restored messages must keep their roles and text; and the conversation must keep its model selection. The merge's conflict rules are asserted directly: given a memory copy older than the local copy and with a different body, the memory copy must win and the local record's messages must be upgraded; given a memory copy older than a newer local copy, the local title and timestamp must survive and the memory's messages must not be adopted. A third group covers the failure path: a `chatstate` payload whose text is not JSON must not throw, must leave the list renderable — in the isolation used, empty — and must not cache the broken payload. The suite also

asserts that the `memoryRestored` key exists in both locale dictionaries and that the status string is built from it.

The bounds are asserted too, and they are why the payload is bounded rather than truncated. With a conversation long enough to exceed the cap, the written payload must be at most 7 900 characters, must be whole JSON, must drop oldest turns rather than newest ones, must end with the newest question, and must contain only real turns — no empty texts, no roles other than user and assistant.

Honest status. This closes the stated limitation — a cleared origin no longer loses conversations, because the memory is a directory on disk — but it is a restoration path, not a synchronisation protocol. There is one writer per record and a last-writer-wins rule keyed on update time; two instances editing the same conversation concurrently resolve by timestamp, and the newer copy wins whole. That is adequate for the failure mode v1 documented, where the second instance simply had no copy, and it is explicitly not a conflict-free design. The storage and atomicity properties that make the memory safe to write this way are given in Section 2.7 and Section 4.2.

2.6 L4: original image bytes for restored turns

The original problem. Page-origin storage has a quota, so v1 kept a 160-pixel thumbnail per image attachment and nothing else. A conversation reopened the next day displayed its pictures correctly and could not send them: resending an old turn replaced the image with an explicit "unavailable" note. The note was honest — better a visible loss than a silent one — but the loss was unnecessary, because the same bytes had already been stored in the memory.

The fix. v2 connects the two. When the composer uploads an image it records the memory's item id on the attachment as `memoryId`; the local-storage copy keeps the id and the thumbnail but, as before, not the bytes. Before a turn is sent, a hydration step walks the conversation and, for each user message with an image attachment that has no inline data, asks the memory for the original with `GET /api/superman/file?id=<memoryId>`, the route v1 already served for previews and downloads. The reply's bytes become base64 on the same attachment object, with the media type so the request body can declare the right image part. Three failure modes fall through to the old behaviour rather than throwing — a response that is not OK, an empty body, and a fetch that rejects — and in each the request builder emits the same unavailable note while the turn is still sent. The file route is unchanged: original bytes with the item's media type, 404 for an unknown id, 410 when the entry exists but its file cannot be read.

The measurement. The chat pipeline suite drives the whole round trip rather than the helper in isolation. A stored image attachment must carry a non-empty memory id; the id must survive the save half and the load half of a history round trip, while the stored copy carries neither bytes nor an inline body — only the id and the pruned flag. Sending that conversation must produce exactly one request to the file route, with the id URL-encoded into the query; the outgoing request body must carry an image part whose decoded byte length equals the original PNG's and whose media type is `image/png`; and the outgoing text must not contain the unavailable note. The answer must still complete normally. The degradation paths are asserted separately: with a file route answering 404, no image part is sent, the note appears, and the turn still finishes; with a route that throws, the same note appears.

Honest status. The item is closed for any image the memory still holds, and is bounded by the capacity policy of Section 2.7: an image evicted by the item or byte cap, or deleted by a user, cannot be hydrated and the turn falls back to the note. Eviction is therefore user-visible in old conversations, which is the correct trade — a bounded memory that silently rewrote history would be worse. The id is also only useful while the index entry survives, and reconciliation removes entries whose file has vanished, after which the route answers 404 and the same fallback applies.

2.7 L5: a capacity policy and start-up reconciliation

The original problem. v1 bounded two things and policed neither properly. Previews were truncated, note counts were limited, duplicates were caught by content hash plus name and each write was atomic; but nothing evicted an old document when a new one arrived, nothing compared the index against the directory it described, and the only way to reclaim space was to delete entries by hand. A store that only grows eventually fails, and the failure is worse than storage exhaustion, because the index and the files it claims to describe can drift apart: an entry whose file was removed outside the application, or a file written by an interrupted operation that no entry mentions.

The fix. v2 gives the store two caps, one eviction path, one repair path and one manual action.

The caps are `maxItems`, 2 000 by default, and `maxBytes`, 512 MiB by default, both configurable on the plugin row and both enforced after every store. Enforcement evicts oldest-first by addition time until both hold, and it is careful about the one case that would otherwise break: the item just stored is protected from its own compaction pass, so a single file larger than the byte cap is still stored. Each eviction removes the index entry and force-removes the file,

increments a counter, and is reported to the caller in the store reply's `evicted` list, so a client can tell the user that something left the memory.

The repair path runs once per store instance, on first load. Reconciliation walks the index and drops every entry whose id is not a non-empty string or whose file does not exist; then it lists the directory and deletes every file no surviving entry references, counting the removals; a file that cannot be removed is left for the next start rather than failing the load. A file an entry still references is never deleted, in either direction of the comparison. If either step changed anything the index is written once; if nothing changed, nothing is written, because a start-up that rewrote the index unconditionally would turn a read into a write.

The manual action is `prune`, which runs the same compaction on demand and reports how many items were removed, how many bytes were freed, and how many items and bytes remain. The `stats` counters grew correspondingly, so the policy is observable rather than implicit: alongside the total, the byte count, the note count and the four category counts, `stats` now reports how many items this instance has evicted and how many orphan files reconciliation removed.

Atomicity is unchanged and is what makes the policy safe. Mutations are serialised through a lock queue, so a read-modify-write cycle cannot interleave with another; index writes are serialised through a separate promise chain, and each writes `index.json.tmp` and renames it over `index.json`, which is atomic on the same filesystem [18]. A crash therefore leaves either the previous index or the new one, never a truncated one. Duplicate detection stays content-hash-plus-name: the same bytes under the same name refresh the existing entry's timestamp instead of adding a second copy, and an item's id is the first sixteen hexadecimal digits of its SHA-256, which makes identity a function of content. Notes remain capped at 500, oldest dropped.

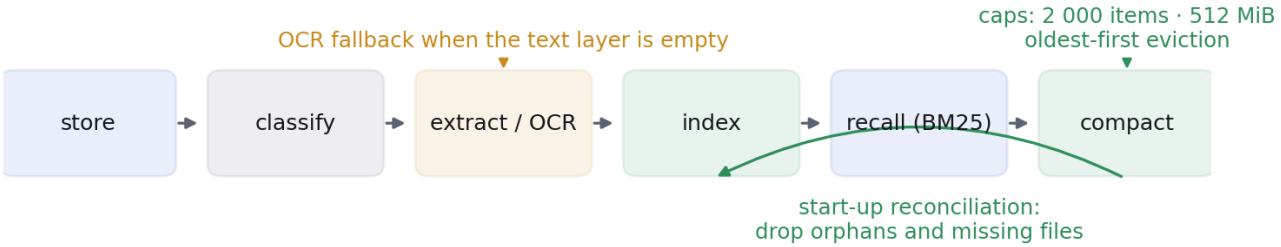


Figure 9. The memory lifecycle in v2: store, classify, extract or OCR, index, recall and compact, with the item and byte caps and the start-up reconciliation step labelled.

Figure 9 draws the lifecycle with the policy attached. The order is not arbitrary: text is extracted before the item is pushed onto the index and before compaction runs, so an evicted item has already done its work, and reconciliation runs before any mutation, so the store never compacts a list it has not yet repaired.

The measurement. The memory host suite exercises the policy adversarially in six groups, all passing. Lowering the item cap and storing four items must evict exactly the oldest, name it in the reply, remove its file and its entry, and report one eviction and a numeric orphan count in `stats`. Storing until the byte cap binds must hold the total after every store rather than only at the end, evict oldest-first, delete those files, and leave exactly as many files as the index references. `prune` under a lowered cap must report removal count, freed bytes, remaining items and remaining bytes matching independently computed values, deleting exactly the evicted files. Reconciliation, given one entry whose file was removed, one unreferenced file and one entry with its file present, must drop the first, delete the second, never delete the third, rewrite the index exactly once and count at least one orphan. Finally, a burst of eight concurrent stores must leave `index.json` valid and complete with all eight entries, no temporary index file behind, and exactly eight files, and the index must still parse after eviction.

Honest status. The item is closed, and what it does not do should be said as clearly. Eviction is oldest-first by insertion time, not relevance-based: a document consulted daily is evicted on the same rule as one never opened. Duplicate detection is content hash plus name, so the same bytes under a different name become two items, which allows byte-identical duplicates to persist. Note bodies are capped at 8 000 characters and the notes array at 500 records. And reconciliation removes data: an orphan file is deleted by design, because a file no entry references is unreachable.

2.8 L6: text extracted when a document is stored

The original problem. v1 had two capture paths into the memory and only one carried text. A chat upload was extracted in the browser before it was sent, so the panel posted both the bytes and the text it had recovered, and recall could match the document's contents. The other two paths posted bytes alone: the Superman panel's own upload control, and the harness capture path, where a `session/event` listener copies attachments the agent's session committed. For those, the memory held a PDF with a filename and nothing else. The limitation is the kind that is easy to miss from the inside: the feature works on the path its author uses most.

The fix. v2 moves extraction into the host half and runs it at store time, for every path. The browser extractor was ported to Node as `superman/pdf.js`, a module that walks the object table, decodes filters with the platform's `DecompressionStream`, parses `/ToUnicode` CMaps, replays content streams and assembles per-page text, with the same tolerance for indirect lengths, inherited resources, object streams and filters declared for bytes that are not in the declared encoding. The port keeps the browser algorithm rather than reimplementing it — apart from two exported caps and their option names it is the shipped browser code — so the two halves cannot drift apart, and a divergence is visible as a difference in a suite that loads both.

The store path consults it in a defined order. If the caller supplied text, that text is used and the reader is not invoked; this is the chat path, and the text is trimmed to the 4 000-character preview limit. If the caller supplied nothing and the item classifies as a PDF, the reader runs with a page cap of 200 and a character cap of 400 000, the extracted text is capped at the same 400 000 characters, and the page count is kept on the item, which also gives the panel a detail string naming pages and size. If the item classifies as code or other, the first 32 KiB are decoded through the same encoding cascade the browser uses and capped at the preview limit. Images keep no text. A parse failure is caught at the call site: the bytes are stored with no text rather than the store failing, which is the same failure direction as v1's `no-text` report. Because extraction now happens on the host, an item also carries its id, name, category, media type, byte count, addition time, source and origin, a detail string, its SHA-256, whether it has text and how long that text is, and, where cheap, image dimensions read from the file header or a PDF page count. The text is returned only by `get` and `store`, so listing the memory does not ship the corpus to the browser; `list` reduces each item to a `hasText` flag.

The measurement. The memory host suite tests the new path with generated documents that isolate one mechanism each, and compares the port against the browser original on real files. It stores a PDF with a literal text layer and requires `hasText` true, a non-zero text length, the expected sentence from `get`, a page count of one and a detail naming the pages; stores a PDF whose text sits inside a `FlateDecode` stream and requires recovery; stores a PDF using a composite Type0 font and requires decoding through its `/ToUnicode` CMap; stores a scanned PDF and requires acceptance with `hasText` false and no exception; and stores an unparseable PDF and requires the bytes on disk to be byte-identical to what was uploaded. A separate check requires the reader never to throw on malformed input.

The agreement checks are the ones that matter for maintenance. The suite loads the browser extractor through the module loader used elsewhere, runs both implementations over every real PDF present in the papers directory and over a set of synthetic PDFs, and requires the texts to be identical in each case; it also verifies that the synthetic fixture really carries the sentence both readers must find, so two empty results cannot pass as agreement. Finally the harness capture path is tested end to end: a PDF attached to a session event must arrive classified as a PDF with its own extracted text, and the same attachment must not be stored twice when the session log repeats it.

Honest status. The item is closed: every path that stores a PDF now stores its text, up to the caps. Two qualifications belong here. The agreement test covers a small and partly synthetic set, so it shows that the port matches on the documents available, not on all documents. And extraction is synchronous with the store, which now includes a parse — for a 56-page prospectus, the 71.8 ms median of Figure 7 — which is the price of recallable text, bounded by the caps.

2.9 L7: browser-level verification, partially closed

The original problem. The v1 suites drive the shipped files under doubles: a stub module loader supplies the browser halves, and a stub React invokes each function component directly instead of rendering it. That executes the component body but cannot observe the document. Nothing in v1 proved that a panel mounts into a DOM, that a keystroke reaches the handler a browser would call, that a file input's change event carries the chosen file, or that an overlay renders where the theme places it; v1 stated this and noted that the deployment had been inspected at the slot, section and prompt level instead.

The progress. v2 adds a workspace-local browser-like toolchain and a driver that uses it. The toolchain is jsdom [9] with React and ReactDOM [10], installed beside the suites so no global dependency is introduced; the driver is `tools/dom-check.mjs`, 884 lines, which builds a real document, installs the plugin stylesheets, mounts each panel with `createRoot` into a host element, and drives real events: input and key events on the composer, clicks on buttons, a `change` event on a hidden file input carrying a `File`, and pointer interaction with the overlay. It covers the Chat panel's mount and composer, sending and streaming, Markdown and math elements in the rendered answer, the memory switch and its persisted state, attachment chips and their store requests, the Stop control, the history column's collapse state, the memory panel's six tabs and their counts, preview, upload and delete, and the toast stack. Its assertions are about nodes in a document — a tag, an attribute, a text node, a class applied after a click — which is exactly the layer several claims in Sections 2.5 and 2.6 are about.

The honest status. The suite is not part of the published tree. Under jsdom the interaction-level paths do not yet converge. React's `act` semantics, the `console.timeStamp` instrumentation the development build expects, and the native value-setter plumbing used to simulate typing all differ from a real browser's, and the difference shows up where the driver has to be most faithful: the Chat panel mounts through `createRoot` into a real document, its stylesheet is written into the document head, and typing reaches the textarea, while the key-driven send does not produce the request the driver waits for and the run stops at the first history-entry click. Until those paths settle the suite is a development artefact: the seed of the browser-level verification v1 lacked, and not evidence for the shipped panels.

Two consequences follow. Pixel-level claims remain unverified: element trees are asserted, appearance is not. And the shipped trees contain six suites, not seven, so the check total in Table 4 is computed without the DOM driver; the 449 checks are the checks that run against the published files.

The distinction between partial and failed is between a tool that does not exist and one that exists and disagrees with its environment on a bounded set of behaviours: the driver already establishes that the panels mount, that the bundles install their stylesheets and that the composer accepts typed input, which the stub React could never show, and what it cannot yet do is finish a turn. Reporting this as closed, or quietly excluding the failing assertions, would be exactly the failure mode this paper is about.

2.10 L8: semantic recall remains open

The original problem. v1's last limitation was that recall is not semantic: scoring is surface similarity over tokens plus recency, so a query that paraphrases stored material without sharing a token with it returns nothing. v1 named the trade: the lexical path is deterministic, cheap and model-free, and it handles Chinese without a segmenter by tokenising characters and character bigrams, but it cannot relate two different words to one meaning. v2 improved the lexical scorer and did not close the semantic gap.

Why it is still open. The reason is a resource, not a design. There is no embedding service on this machine, and a dense retriever cannot be evaluated without one: that would mean embedding the corpus, embedding a query set and scoring the ranking against labels, and none of those three artefacts exists here. Adding an unmeasurable retriever and asserting that it helps would be the opposite of this paper's method.

What an embedding path would need. Four things, in dependency order. First, a service: the host half can call a local model server or a remote API, either way gaining a network dependency the memory does not have today, which is why the honest architecture is another optional companion bundle — detected, used when present, absent without breaking. Second, an index: vectors for items and records beside `index.json`, keyed by item id, with approximate-nearest-neighbour search on top; the standard choices are exhaustive similarity search [19] and a graph-based index [20], and for a memory of at most a few thousand documents exhaustive search over normalised vectors would suffice and avoid a third-party index format. Third, retrieval design: a single dense vector per document retrieved by inner product is the simplest option and the one the literature establishes first [21], [22], while a later-interaction or reranking stage over the top candidates improves ordering at a cost in computation [23]; the composite shape recent work favours [24] is what the memory already has, because its output is a digest prepended to a question or injected into a prompt, so only the ranking stage would change — and returning more text to the model is not automatically more useful [5]. Fourth, the arithmetic of maintenance: vectors must follow the lifecycle of Figure 9, written on store, deleted on delete or eviction, repaired when reconciliation removes an entry — a third artefact that needs its own reconciliation rule.

What v2 did to prepare the ground. The recall seam is one function. `recall(query, limit)` takes the query text and a limit and returns scored matches with a snippet; the command route calls it for the `recall` action and renders the digest from what it returns; the composer posts the question and prepends the returned digest; the harness reads the

digest through a prompt section whose text is a function evaluated at every prompt assembly. Replacing the ranking inside that one function — or replacing it with a dispatcher that prefers a dense score when vectors and a service are available and falls back to BM25 when they are not — requires no change to the route, the panel, the prompt section or the caller contract. The item is open but not blocking: the seam is narrow and the fallback is already shipped and measured.

Honest status. Open, and recorded as open in Table 2. The claim is the weaker one that is supported: the lexical scorer's two documented defects were fixed and verified, and the semantic ceiling above it was not touched. The motivation for a durable memory that outlives a session is well established — an operating-system-like memory hierarchy [25], an agent architecture with a memory stream and reflection [26], and the broader line of acting and tool-using agents [3], [4] — but the distinguishing property here is the storage rather than the retrieval: the memory is a directory of ordinary files with a JSON index, so the agent reads primitives directly and the user can inspect or delete them with ordinary tools. That choice makes the lexical path acceptable as a floor and a vector index a local addition rather than a re-architecture.

3 Discussion

3.1 What a closure pass teaches

First, a fix belongs where the behaviour lives, and that determines what a user must do to receive it. Six limitations were closed by changing code, and the changes split along the plugin boundary: three live in browser halves — the OCR fallback call, history hydration and image restoration — and reach a running page through the hot-reload watcher without a restart, while three live in host halves — BM25 recall, compaction with reconciliation, and store-time extraction — and require a process restart. Table 1 records that split as an operational fact rather than an implementation detail.

Fifth, the two items that remain open are not open for the same reason. L7 is a tooling problem with a visible endpoint: the driver exists, its assertions are written, and what stands in the way is a bounded set of jsdom and React environment differences, so it is a work item. L8 is not: it needs a decision about dependencies and an evaluation that does not exist, because there is no labelled query set for this corpus and no embedding service to produce vectors for one. Grouping the two as "unfinished" would obscure that one can be scheduled and the other has to be decided.

3.2 Threats to validity

One machine, one platform. Every measurement was taken on a single macOS machine and a single harness profile. The consequence is sharpest for OCR, whose engine is the Vision framework with PDFKit rendering: the L1 fix is bound to macOS, and elsewhere the companion bundle is absent and the chat half falls back to the `no-text` report. There is no second OCR backend, and the documented behaviour without the bundle is the old behaviour.

A small and unrepresentative corpus. The extraction measurements use fourteen real documents, nine of them vector art from one vendor that contribute nothing but a refusal; two Chinese prospectuses dominate the character totals; the names are aliases. The OCR measurements are three cases, one per failure mode. The reported medians describe these files on this machine, not a distribution over document kinds. The GPT-4 technical report appears in the corpus only as one stored sample [27] and carries no design claim.

Yield is not accuracy. The OCR results are characters and Han characters recovered, not a character error rate, because no ground-truth transcript was produced. A yield measure answers whether the path returns a page's text at all, which is what the limitation was about; it does not answer how many characters are wrong, which would need a reference transcript and a distance metric such as edit distance [28] over a sequence-decoding formulation [29]. The paper argues from plausibility — a layer with 9 Han characters out of 9 423 cannot be a correct reading of a Chinese brochure — rather than from a scored reference.

A constructed retrieval corpus. The BM25 checks use three short documents designed to expose the two documented defects. They establish properties and are not a benchmark: no precision, recall or ranking metric was computed over a labelled collection, because none was built. Stronger claims about dense retrieval [21], [22], [23] or retrieval-augmented generation [24] are outside what this paper measured, and the transformer machinery those methods rest on [30], [31], [32] was not exercised here.

Self-authored verification. The 449 checks are assertions in our own suites, so their count measures coverage of the paths we chose to test, not correctness in general, and no external conformance suite exists for these bundles. The suites drive shipped files under doubles, which is stronger than driving a copy but weaker than driving a real

browser and a real provider: the `/api` trust fence is the host's guarantee rather than something the suites exercise, provider output is canned, and visual conclusions are out of reach. Line counts, check counts and timings are also snapshots of one tree at one moment.

No end-to-end model evaluation. Nothing here measures whether the memory changes an answer for the better. The digest reaches the model, the tests assert its content, and the value of that content to a model is not measured. That is the gap L8 would have to cross.

3.3 The status of L7 and L8

L7 — partially closed. What exists is a jsdom and React toolchain in the workspace, an 884-line driver, real `createRoot` mounts, real DOM events, and passes on the mount, stylesheet and typing paths. What does not is the send path and the history interaction, which do not converge under jsdom, so the suite is not in the published tree and its assertions are not counted in the 449. What would close it: those environment differences resolved so that a keystroke reaches the send handler and a click on a history entry swaps the thread, followed by inclusion in the published tree and in the totals.

L8 — open. What exists is a recall seam that is one function, a BM25 scorer behind it with verified properties, an unchanged contract, a deterministic fallback that needs no model, and a stated design for a dense path. What does not is any embedding service, vector index, labelled query set or measured ranking quality. What would close it: a service available to the host half; vectors written on store and removed on delete or eviction; a labelled set of queries over a real corpus with a reported ranking metric for both paths; and a fallback that keeps recall working when the service is unreachable.

4 Methods

4.1 The harness plugin model

DSH is a Node process whose interface is a browser application served from a loopback address; extensions are Cordis plugins [2]. A bundle packages a host half (a Node ES module exporting `apply(ctx, config)`), an optional client half (an ES module served to the page), a manifest and a patch inserting one host row into a profile, and installation goes through the harness's own bundle installer without modifying a harness source file. Interface contribution is additive — a plugin declares a slot with `ctx.slots.inject` and contributes a component under an id of its choosing — so the suite occupies `sidebar.panellist` (ids `conversation` at order −20, `chat` at −10, `superman` at −5), `main`, `settings.section` (id `arxiv`) and `shell.overlay` (id `superman.toast`) without displacing a shipped entry. Transport uses the authenticated Fetch channel rather than a new socket: a plugin registers an exact path with `ctx.connection.fetch.register`, inheriting the host/origin trust fence and the browser-session credential. The five paths are `POST /api/chat-mode/stream`, `POST /api/superman/api`, `GET /api/superman/file?id=`, `POST /api/superman-ocr/api` and `GET /arxiv/search`, the last a same-origin proxy for the arXiv Atom feed [33]. Every reply is one JSON envelope, `{ok, value}` or `{ok: false, error: {code, message}}`, with the status derived from the code (400 for `bad-request`, 404 for `not-found`, 413 for `too-large`, 499 for `aborted`, 502 for `ocr-failed`, 504 for `ocr-timeout`, else 500). Chat answers stream as Server-Sent Events [34], one frame per chunk written to a `ReadableStream` and reassembled by the page on the double newline, with the request's abort signal propagated into the provider call.

Three extension points carry the memory: a `systemPrompt.section` named `superman-memory`, whose `text` is a function so the digest is recomputed at every prompt assembly; a `session/event` listener that captures Harness uploads and writes a task record at `turn/end`; and the optional OCR route. A client half may import React [10] and nothing else from the harness's client packages, so the PDF reader, encoding detector, math translator and Markdown renderer live inside the bundles — mathematics as MathML [35] from LaTeX source [36], laid out with a system math font, preferably STIX Two [37], rather than KaTeX [38] or MathJax [39], and attachments decoded through the Unicode character set [40] and the WHATWG encoding labels [41]. Client halves reload through a 500 ms watcher without a restart; an edited host half needs one.

Table 6. Memory and OCR API actions added or changed in v2.

module	route	action	status	note
superman	/api/superman/api (+ GET /api/superman/file?id=)	list	carried	unchanged contract, stats now carry eviction counters
superman	/api/superman/api (+ GET /api/superman/file?id=)	store	changed in v2	PDF text is extracted on the host at store time
superman	/api/superman/api (+ GET /api/superman/file?id=)	note	carried	unchanged
superman	/api/superman/api (+ GET /api/superman/file?id=)	get	carried	unchanged contract, returns the extracted text
superman	/api/superman/api (+ GET /api/superman/file?id=)	delete	carried	unchanged
superman	/api/superman/api (+ GET /api/superman/file?id=)	recall	changed in v2	BM25 (k1 1.2, b 0.75) with IDF replaces the flat additive score
superman	/api/superman/api (+ GET /api/superman/file?id=)	prune	new in v2	manual compaction pass; returns what the policy removed
superman-ocr	/api/superman-ocr/api	health	new in v2	{ready, bin, compiled, version}
superman-ocr	/api/superman-ocr/api	ocr	new in v2	{text, pages, characters} from the Vision engine

4.2 The storage schema and atomicity

The memory is a directory `<root>`, by default the harness home's `superman` folder, holding `index.json` and `files/<id>`. The index declares version 1 and holds items and records. An item carries an id, name, category (`pdf`, `image`, `code` or `other`), media type, byte count, addition time, source, an optional origin, a detail string, its SHA-256, a `hasText` flag and text length, optional image dimensions, an optional page count and the text. A record carries an id, kind (`task` or `conversation`), title, body, source, an optional session id, timestamps and up to twelve tags. Bodies are capped at 8 000 characters and item text at 400 000; `text` is returned only by `get` and `store`, while `list` reduces it to `hasText`.

An item's id is the first sixteen hexadecimal digits of its SHA-256, so identity is a function of content. Duplicate detection is the pair (digest, name): the same bytes under the same name refresh the existing entry, while the same bytes under a different name become a second item, which keeps the directory legible to the agent reading it. Image dimensions are read from the file's own header — PNG from its IHDR fields, GIF from the logical screen descriptor, JPEG by walking segment markers to a start-of-frame marker, WebP from the extended-format chunk — so a screenshot is identifiable without being opened and no image library is required.

Writes are serialised twice over: whole mutations run through a lock queue, so a read-modify-write cycle cannot interleave, and index writes run through a separate promise chain, each writing `index.json.tmp` and renaming it over `index.json`. Rename within a directory is atomic, so a crash mid-write leaves either the previous index or the new one and never a truncated one [18]. An index whose version or arrays do not match is ignored and rebuilt from the files on the next write, and records are capped at 500, oldest dropped.

4.3 The BM25 formulation

Each item and record becomes one document, represented as a weighted term-frequency map built from `[text, weight]` pairs: the name, or a record's title, weighs 3 and the body weighs 1. Tokenisation lowercases, splits into Latin words of two to forty characters, and collects CJK characters each followed by the bigram it forms with its successor; a query is tokenised the same way and de-duplicated. With N the number of documents, n(t) the number containing term t, f(t, D) the weighted frequency of t in D, |D| the weighted length of D and avgdl the mean weighted length over the corpus:

```
IDF(t) = ln( 1 + ( N − n(t) + 0.5 ) / ( n(t) + 0.5 ) )

score(D, Q) = Σ over t in Q of IDF(t) · f(t, D) · (k1 + 1) / ( f(t, D) + k1 · ( 1 − b + b · |D| / avgdl ) )
```

with k1 = 1.2 and b = 0.75. Documents scoring zero are discarded, the rest sorted, a recency bonus of at most 1.5 fading linearly over thirty days is added, the list is sorted again by the sum, and it is truncated to the requested limit of six by default and twenty at most, with a snippet of at most 200 characters around the earliest matching token. Because frequency and length are both weighted, the field weight acts as a prior on how much a term in a name counts in the normalisation as well as the numerator; that is why a name-only match can outrank a body match in a much longer document.

4.4 The OCR pipeline

The engine is a Swift program with no package dependencies. For each page it renders the media box with PDFKit into an RGB bitmap at two device pixels per PDF point — filling white first, scaling the render down when the longest edge would exceed 4 000 pixels — and recognises it with a `VNRecognizeTextRequest` configured for `.accurate` recognition, languages `zh-Hans` and `en-US`, and language correction enabled. Observations are sorted into reading order by the line's vertical centre, with a tolerance of 0.015 in normalised coordinates before the left edge

breaks the tie. Each page is emitted as a `## page N` marker followed by its lines as soon as it is recognised, so a timeout preserves finished pages; a page producing no bitmap or whose recognition throws is reported on standard error and skipped; and the program exits non-zero if the document cannot be opened, is encrypted, has no pages, or yields no rendered page. The classical alternative is to embed a general OCR engine such as Tesseract [42]; the platform engine was chosen because it is already present, ships no model data, and reads Simplified Chinese and English without additional training.

The bundle exposes one route and performs the compile-on-first-use described in Section 2.3: `swiftc -O <source> -o <staging> -framework Vision -framework PDFKit -framework AppKit`, with the clang and Swift module caches redirected under the work directory, the staging binary renamed into place and marked executable only after a successful exit, and its version probed for the health reply. Configuration comes from the plugin row (`workDir`, `bin`, `source`, `swiftc`, `maxPages`, `timeoutMs` and `scale`), and limits are exported from the module so callers and the offline suite read the same constants: 32 MiB of PDF, a base64 ceiling derived from it, 400 000 characters of returned text, and a 1 200-character tail of a failing child's standard error carried into the error message. The chat half calls the route with a 600-second abort timer, only after its own extractor has returned an empty text layer.

4.5 The compaction policy

Two caps govern the store: `maxItems`, 2 000 by default, and `maxBytes`, 512 MiB by default, both configurable on the plugin row. After each store the policy evicts the oldest items by addition time until both caps hold, never evicting the item just stored, so one oversized upload still stores; each eviction removes the index entry and force-removes the file, increments a counter and appears in the store reply's `evicted` array. Termination is immediate: each iteration removes exactly one item from a finite list, and the loop stops when neither cap is exceeded or only the protected item remains — the same invariant-and-variant shape as the classic partitioning proof [43]. Once per store instance, on first load, reconciliation matches the index against the directory: an entry whose id is not a non-empty string, or whose file does not exist, is dropped; a file no surviving entry references is deleted and counted; a file that cannot be removed is left for the next start; and a file an entry does reference is never deleted. The index is rewritten only if one of those steps changed something. `prune` runs the same compaction on demand and reports items removed, bytes freed and what remains, and `stats` reports the category counts, total and bytes, the note count, and the policy's own counters.

4.6 The offline verification methodology

The suites live beside the bundles, run with Node, and import the shipped files rather than copies. Browser halves load through a stub `window.__ModuleLoader__` that captures the factory call, and the factory is invoked with a stub `require('react')` providing only the element factory, memo, fragment and the state, reference and effect hooks and invoking function components directly rather than rendering them, so component bodies execute with real hooks and props and no reconciler is needed; the document, navigator, timers (unref'd so a poller cannot hold the process), an in-memory `localStorage` with an optional quota, and a `fetch` answering canned payloads are doubles. Host halves run against a fake Cordis context implementing injection, effects, events, service lookup and route registration, with the memory root created by `mkdtemp` and removed afterwards. Real inputs are used wherever they exist: the fourteen-document corpus, real encoding bytes, generated PDFs carrying literal, FlateDecode and Type0 text layers, and the real Swift source alongside a fake executable driving the compile, timeout and cleanup branches. Real therefore means the plugin sources, contract validation, state machines, PDF extraction, encoding detection, the index and its files, SSE reassembly, request URLs and bodies, and element trees; doubled means the browser environment, the provider and the authentication fence; and not proven means pixel-level appearance, real-browser layout, real model output, and visual conclusions.

4.7 The browser-level DOM harness

The driver builds a jsdom document [9], installs the plugin stylesheets into its head, and mounts each panel with `createRoot` from ReactDOM [10] into a host element it creates and appends. Interactions are dispatched as real events: input and key events on the composer, `click` on buttons, `change` on a hidden file input carrying a constructed `File`, and pointer interaction with the overlay, with React updates flushed inside `act` where the environment allows. The toolchain is installed locally beside the driver so the bundles and published suites acquire no dependency, and the driver is not invoked by any published suite.

4.8 Reproduction

The published suites and the metric script run from the repository root:

```
node tools/chat-mode-check.mjs
node tools/chat-mode-format-check.mjs
node tools/superman-host-check.mjs
node tools/superman-check.mjs
node tools/arxiv-smoke.mjs
node tools/superman-ocr-check.mjs
python papers/v2/tools/metrics.py
```

Each suite exits non-zero if any assertion fails and prints the failing label with its observed value. The real-document checks inside the chat pipeline suite need PDFs on the machine, searched for in a default downloads directory that `DSH_TEST_CORPUS` can override, and are skipped when no corpus is present, so the total on a bare machine is lower than the 449 of Table 4. The OCR suite compiles the Swift tool on first run and caches the binary, and `papers/v2/tools/metrics.py` re-runs the suites, the reader comparison and the OCR measurement and writes the measurement file.

Data availability

The measurement data behind every figure and table are in `papers/v2/data/metrics.json`, regenerated by `papers/v2/tools/metrics.py` from the shipped code and the local corpus. The corpus is not redistributed: it consists of fourteen documents from private sources, identified here and in the data file by aliases (`brochure.pdf`, `prospectus-2025.pdf`, `prospectus-2026.pdf`, `contract-2027.pdf`, `document-B.pdf` and nine unaliased vector-art documents), and the reader's per-file output is reported for each: bytes, median extraction time, characters, CJK characters, noise characters, page count and per-page density. The OCR cases are three documents held locally; their text-layer and recognised character counts, Han counts and page counts are reported in Table 5. The verification suites described in Section 4.6 are the other data set: each suite is executable and reports its own assertions, and their labels are the record of what was tested.

Code availability

The source is organised as two trees in one repository: `v1/` is the code exactly as the v1 article published it, and `v2/` is the tree this paper describes, so the delta can be read directly. The v2 host-side PDF extractor is `superman/pdf.js` (799 lines), the OCR bundle is `superman-ocr/index.js` (383 lines) with its Swift engine at `tools/ocr-pdf.swift`, the changed host half of the memory is `superman/index.js` (908 lines) and the changed browser half is `chat-mode/client.js` (4 281 lines). The published verification code is the six suites in `tools/` (2 792 lines) with the loader they share; the DOM driver of Section 2.9 is not in the published tree and is not covered by this claim. The code is version-controlled with Git [44], published through the GitHub API surface for repository contents [45], and released under the MIT licence [46]. Each bundle installs through the harness's own bundle installer from its package directory; the OCR bundle is optional and its absence is a supported configuration.

Acknowledgements

We thank the DeepSeek Harness and Cordis projects for the extension framework this work builds on, the maintainers of the specifications cited below, and arXiv for an open search interface and for cross-origin access to paper PDFs.

Author contributions

A.Z. designed the closure programme, implemented the host-side extractor, the BM25 scorer, the compaction and reconciliation policy, the OCR bundle and the Swift engine, wrote the browser-side changes for history restoration, image hydration and the OCR fallback, ran the measurements and drafted the manuscript. Z.P. designed and built the v1 suite this paper starts from, and reviewed the v2 changes and the measurement methodology. DeepSeek Harness Contributors provided and reviewed the harness extension surface. All authors approved the final manuscript.

Competing interests

The authors declare no competing interests.

References

1. DeepSeek Harness — https://github.com/deepseek-ai/deepseek-harness
2. Cordis plugin framework — https://github.com/cordiverse/cordis
3. S. Yao et al., ReAct: synergizing reasoning and acting in language models, ICLR 2023.
4. T. Schick et al., Toolformer: language models can teach themselves to use tools, NeurIPS 2023.
5. N. F. Liu et al., Lost in the middle: how language models use long contexts, TACL 2024.
6. Apple Inc., PDFKit framework, developer documentation.
7. Apple Inc., Vision framework — VNRecognizeTextRequest, developer documentation.
8. S. Robertson and H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Found. Trends IR 3(4), 2009.
9. jsdom — https://github.com/jsdom/jsdom
10. React — https://react.dev/
11. ISO 32000-1:2008, Document management — Portable document format — Part 1: PDF 1.7.
12. Adobe Systems, PDF Reference, 6th edn (FlateDecode, ToUnicode CMaps), 2006.
13. ISO 19005-1:2005, PDF/A-1 (archival PDF).
14. S. E. Robertson, S. Walker, S. Jones, M. M. Hancock and M. Beaulieu, Okapi at TREC-3, TREC 1994.
15. G. Salton and C. Buckley, Term-weighting approaches in automatic text retrieval, Inf. Process. Manag. 24(5), 1988.
16. K. Sparck Jones, A statistical interpretation of term specificity and its application in retrieval, J. Doc. 28(1), 1972.
17. WHATWG, HTML Living Standard, § Web storage (localStorage).
18. M. Kleppmann, Designing Data-Intensive Applications, O'Reilly, 2017 (atomic replace, idempotence).
19. J. Johnson, M. Douze and H. Jégou, Billion-scale similarity search with GPUs (FAISS), IEEE Trans. Big Data, 2019.
20. Y. A. Malkov and D. A. Yashunin, Efficient and robust approximate nearest neighbour search using HNSW, IEEE TPAMI, 2018.
21. K. Karpukhin et al., Dense passage retrieval for open-domain question answering, EMNLP 2020.
22. N. Reimers and I. Gurevych, Sentence-BERT, EMNLP 2019.
23. O. Khattab and M. Zaharia, ColBERT, SIGIR 2020.
24. P. Lewis et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, NeurIPS 2020.
25. C. Packer et al., MemGPT: towards LLMs as operating systems, 2023, arXiv:2310.08560.
26. J. S. Park et al., Generative agents: interactive simulacra of human behavior, UIST 2023.
27. OpenAI et al., GPT-4 Technical Report, 2023, arXiv:2303.08774 (used only as a corpus sample).
28. V. I. Levenshtein, Binary codes capable of correcting deletions, insertions and reversals, 1966 (edit distance, for OCR error discussion).
29. L. R. Rabiner, A tutorial on hidden Markov models, Proc. IEEE 77(2), 1989.
30. A. Vaswani et al., Attention is all you need, NeurIPS 2017.
31. J. Devlin et al., BERT, NAACL 2019.
32. T. Brown et al., Language models are few-shot learners, NeurIPS 2020.
33. arXiv API user manual — https://info.arxiv.org/help/api/user-manual.html
34. WHATWG, HTML Living Standard, § Server-sent events.
35. W3C, MathML Core, W3C Recommendation.
36. D. E. Knuth, The TeXbook, Addison-Wesley, 1984.
37. STIX Two fonts — https://github.com/stipub/stixfonts
38. KaTeX — https://katex.org/
39. MathJax — https://www.mathjax.org/
40. The Unicode Consortium, The Unicode Standard, Version 15.
41. WHATWG, Encoding Standard (GBK, Big5, Shift_JIS, EUC-KR decoders).
42. R. Smith, An overview of the Tesseract OCR engine, ICDAR 2007.
43. C. A. R. Hoare, Proof of a program: FIND, 1971.
44. Git project — https://git-scm.com/
45. GitHub, REST API for repository creation and contents.
46. MIT License — https://opensource.org/license/mit

Appendix A. Memory and OCR API actions

`POST /api/superman/api` takes a JSON body with an `action` field. `list` returns the items (each reduced to a `hasText` flag), the records, the statistics and the memory root; `store` takes a name, an optional media type, base64 bytes, optional source and origin, and an optional pre-extracted text, and returns the item plus whatever the capacity policy evicted for it; `get` returns one item with its text or a record projected into the item shape; `delete` accepts an item id or a record id; `note` creates or replaces one record; `recall` returns scored matches with snippets plus the rendered digest; `prune` runs the compaction policy on demand. `GET /api/superman/file?id=` returns an item's original bytes with its media type, 404 for an unknown id and 410 when the entry exists but its file cannot be read. The OCR

bundle exposes `POST /api/superman-ocr/api` with `health` and `ocr`, on the same envelope and code-to-status mapping. Table 6 lists what changed in v2.

Appendix B. Verification suite inventory

Six suites are published. The chat pipeline suite drives route registration and validation, SSE framing and cancellation, multi-turn history, attachment inlining, the memory wiring including uploads, question digests, turn mirroring, the switch and the eight-second timeout, the OCR fallback and its failure path, image hydration with its 404 and throwing routes, restorable-payload capping, the history merge, and provider failure paths. The formatting suite drives LaTeX-to-MathML translation and the sixteen-expression battery, Markdown structure, protection of dollar amounts, encoding detection over real bytes, and HTML tag stripping. The memory host suite drives classification and image-size parsing, store and index writing, the `get` and `file` routes, record upsert, recall and digest rendering, deletion, harness capture and turn records, store-time extraction with its agreement checks against the browser reader, the BM25 properties, and the compaction and reconciliation policy. The memory panel suite drives the entry and its order, the brain icon, dictionary-key parity, the six folder tabs, cards, preview, upload and delete requests, notices and deduplication, and error and empty degradation. The arXiv suite drives the search route's branches and pacing, Atom parsing and rendering, BibTeX copy, and the three save-to-memory paths. The OCR suite drives the route, the envelope, validation and size limits, page-budget clamping, text caps, timeout and abort handling, temporary-file cleanup, compile-on-first-use and cached-binary adoption, compile failure and retry, spawn-failure and timeout reporting, the manifest and patch, locale completeness, and a path that compiles and runs the real tool. Table 4 lists the suites and their counts; Table 3 lists the code they drive.

Appendix C. Corpus and OCR measurements

The corpus measurements are the v1 measurements: fourteen real documents, 199 942 characters, 67 235 Han characters, 138 pages, zero replacement and C1 control characters, five documents with a text layer and nine `no-text` reports, with medians of 34.7 ms for the 19-page document, 71.8 ms and 71.3 ms for the two large Chinese documents, 11.7 ms for the 7-page contract, 7.3 ms for two pages, and 0.2–0.8 ms for the nine textless files; Figure 6 and Figure 7 plot them. The OCR cases add a text-layer character count and a recognised character count with its Han count: 0 → 3 411 characters with 979 Han for the first rasterised page, 1 → 1 341 with 1 072 Han for the second, and 9 423 with 9 Han → 11 333 with 66 Han for the 19-page brochure whose embedded CMap is damaged. Figure 5 plots the pairs and Table 5 gives each case with its page count and cost.

Appendix D. Limitation-to-fix register

L1: closed by the OCR bundle and the chat fallback, evidenced by the 0 → 3 411 comparison and the OCR suite. L2: closed by BM25 at k1 = 1.2 and b = 0.75 over weighted fields, evidenced by the rare-term and IDF checks. L3: closed by the `chatstate:` payload and the start-up merge, evidenced by the fresh-origin, upgrade, no-overwrite and malformed-payload checks. L4: closed by the `memoryId` and hydration path, evidenced by the byte-carrying request, the media type, the absence of the note and the 404 fallback. L5: closed by the caps, oldest-first eviction, reconciliation and `prune`, evidenced by the eviction, byte-cap, prune, reconciliation and burst checks. L6: closed by the host-side extractor, evidenced by the per-mechanism checks, the two-reader agreement checks and the harness-capture check. L7: partial. L8: open. Table 2 is the same register in tabular form and Figure 2 draws it as a matrix.

Appendix E. Reproduction and repository layout

The repository holds two independent trees and the paper material: `v1/` (the original release), `v2/` (the current tree, the same packages plus `superman-ocr`), `papers/` (the v1 articles with their data, figures and tools) and `papers/v2/` (this article with its parts, data and figures). Installation points at `v2/<package>` for current behaviour and `v1/<package>` to reproduce the original, and the OCR bundle is optional. The six published suites and the metric script run from the repository root with `node v2/tools/<suite>.mjs`; the DOM driver runs the same way from the development workspace but is not shipped. The corpus for the real-document checks is located through `DSH_TEST_CORPUS`, and figures and tables are generated from `papers/v2/data/metrics.json`, which `papers/v2/tools/metrics.py` rewrites.